

별무리

iOS 온디바이스 AI 개발 / 김민석

스마트폰에서 캐릭터와 대화하고 알람과 일정 같은 기기 기능을 연결하는 iOS 앱입니다. 소형 언어 모델을 기기 안에서 실행하며, 저장한 대화 내용을 검색해 이후 답변에 사용합니다.

기간	2026.05 - 현재
참여 과정	AI-SW 마에스트로 서울 제17기
담당	온디바이스 AI 개발
진행 상태	TestFlight 사용자 테스트 중, 정식 출시 준비

전체 구조와 사용 기술

구성 요소	기술	역할
클라이언트	Unity	사용자 입력과 세션 정보 전달, 생성 중인 답변 표시
AI 요청 처리	Swift	대화와 도구 요청 분기, 기억 검색과 모델 입력 구성, 답변 조각 반환
언어 모델 추론	Gemma 4 E2B / LiteRT-LM	기기 CPU에서 대화 답변과 도구 호출 인자 생성
임베딩과 분류	EmbeddingGemma / MLP	실행 의도 분류, 도구 선택과 기억 검색에 쓸 문장 벡터 생성
장기 기억	SQLite	저장 대상 발화와 벡터 보관, 현재 질문과 관련 있는 기억 검색

텍스트 생성과 임베딩은 기기에서 실행. 캐릭터 발화의 듣기 기능은 백엔드에서 음성 합성

담당 구현

- Unity 요청을 C API와 Swift 브릿지를 거쳐 LiteRT-LM 모델 호출로 연결
- EmbeddingGemma와 SQLite 기반 장기 기억의 저장과 검색 구현
- 실행 의도 분류와 도구 선택, KV 캐시 재사용과 파일 저장, 복원 구현
- 분류기 평가, 추론 결과 비교, 아이폰 자원 측정을 위한 평가 환경 개발

소스 코드: <https://github.com/mornye-minor-gallery/PetAI-AI>

기술 문서: <https://pysunn.me/docs-beolmuri-ai/>

일반 대화를 도구 실행 요청으로 오인하는 문제

문제

- 여러 도구 설명을 한 번에 넣은 프롬프트에서 소형 모델이 일반 대화도 실행 요청으로 판단
- 임베딩 유사도로 바꾼 뒤에도 일반 대화 24건 중 11건이 호출 후보로 분류

원인과 분류 방식

- 기능 문의, 과거 경험, 인용문도 도구 설명과 유사도가 높아, 실행 의도와 도구 종류를 분리해 판단하도록 구성
- EmbeddingGemma의 768차원 벡터를 입력받는 경량 MLP를 도구 선택 앞에 추가

학습 데이터 수정

- 초기 MLP의 일반 대화 오인을 줄이려고 비실행 사례를 보강했지만, 실제 실행 요청 탐지가 163건에서 118건으로 감소
- 같은 기능의 실행 요청과 비실행 문장을 함께 학습하도록 바꿔, 실제 실행 요청 탐지를 160건으로 회복

학습 구성별 호출 후보 판정

실제 실행 요청 168건, 일반 대화 24건으로 비교

학습 구성	일반 대화 오인	실제 요청 탐지
초기 MLP	16/24	163/168
비실행 사례만 보강	2/24	118/168
기능별 실행/비실행 쌍 보강	2/24	160/168
위 구성에 한국어 비실행 사례 추가	1/24	159/168

마지막 두 구성은 모두 182/192 정답. 비실행 사례를 더한 최종 구성은 일반 대화 오인 1건 감소, 실제 요청 누락 1건 증가

적용

- MLP를 통과한 발화만 도구별 대표 벡터와 비교. 선택한 도구 하나의 정의를 모델에 전달해 JSON 인자 생성
- 한 요청에서 도구 하나만 선택. 여러 도구가 기준을 넘으면 요청을 나누도록 안내

개발 중 열람한 평가셋을 재평가한 결과. 호출 후보 판정만 평가했으며, 새 평가셋과 실기기 전체 도구 실행은 추가 검증 대상

반복 대화의 대기 시간을 줄인 KV 캐시 재사용

사용자 피드백과 수정

- TestFlight 사용자에게 첫 답변이 늦게 나온다는 피드백을 받아 응답 대기 시간 개선
- 매 요청에서 같은 고정 프롬프트를 다시 처리하는 비용을 줄이기 위해 CachedSession으로 메모리의 KV 캐시를 다음 요청에 재사용

메모리 KV 캐시 재사용 측정

iPhone 17, 같은 5턴 대화를 조건별 3회 실행. 총 30개 요청 비교

첫 출력 대기 시간	요청마다 새 세션	캐시 재사용
전체 평균	4.920초	1.274초
후속 요청 평균	4.872초	0.446초

전체 평균 74.1% 감소, 후속 요청 평균 90.8% 감소. 답변 15쌍 모두 일치

앱 재실행 후 재사용을 위한 파일 저장과 복원

남은 문제와 엔진 수정

- 앱을 종료하면 메모리의 캐시가 사라져, 재실행 시 같은 고정 프롬프트를 다시 계산해야 했음
- 사용하던 LiteRT-LM의 공개 C API에 상태 저장과 복원 기능이 없어 엔진을 포크하고 CPU 세션 상태 전송 API 추가
- 엔진이 KV 텐서, 처리 위치, 토큰 ID, 샘플러 상태를 전송하고 Swift가 파일 저장과 복원, 모델 정보와 체크섬 검사 담당

현재 검증 범위

- Mac CPU의 별도 프로세스에서 복원한 뒤, 다음 대화와 동적 입력 변경 조건의 탐욕적 디코딩 출력이 재계산 결과와 일치
- 아이폰의 파일 저장과 복원 시간, 메모리 피크는 미측정